Today's Plan

Career Spotlight

Web Development Workshop

Web Development Workshop

https://go.iu.edu/H9evyd

ACCESS Operations: NSF #2138307

Flanker Task

You will see a row of five symbols.
Respond to the **center arrow only** — ignore the others.

`<<<<<`	Press LEFT — all arrows match
`>>>>>`	Press RIGHT — all arrows match
`>><>>`	Press LEFT — center arrow is <
`--<--`	Press LEFT — ignore the dashes

Press ← / → on a keyboard, or use the on-screen buttons on mobile.

Respond as **quickly and accurately** as possible.

Your name (optional)

Anonymous

https://go.iu.edu/PmAn7N

Web Development: Why?

Low Barrier of Entry

- Simple tooling (All you need is a text editor)
- HTML & JavaScript are relatively easy and fun to learn

It's Everywhere

- Runs on any computer/mobile with a browser
- You'll almost always encounter some form of web programming problem to solve
- Off the shelf tools are great but you might still need some JavaScript knowledge to customize

Vue.js Basics

A Vue Component is Like a House

Every Vue component lives in a single `.vue` file with three sections — think of them like the parts of a house.

`<template>`

The **structure** — the objects in the house: a door, a lamp, a window

`<script>`

The **behavior** — the actions: turning on a light, opening the door

`<style>`

The **appearance** — colors, sizes, and how everything looks

The Three Sections

```
<template>
  <door @openDoor="openDoorInstructions()"></door>
  <window class="bayWindow"></window>
  <lamp></lamp>
</template>

<script>
  function openDoorInstructions(){
    unlatch();
    swingOpen();
    stopSwing();
  }
</script>

<style>
  .bayWindow {
    frame: white,
    opacity: clear,
    width: 4feet,
    height: 6feet
  }
</style>
```

<template>

The objects — door, window, lamp

<script>

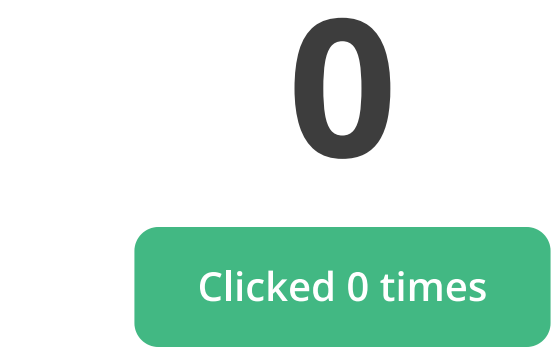
The behavior — open the door

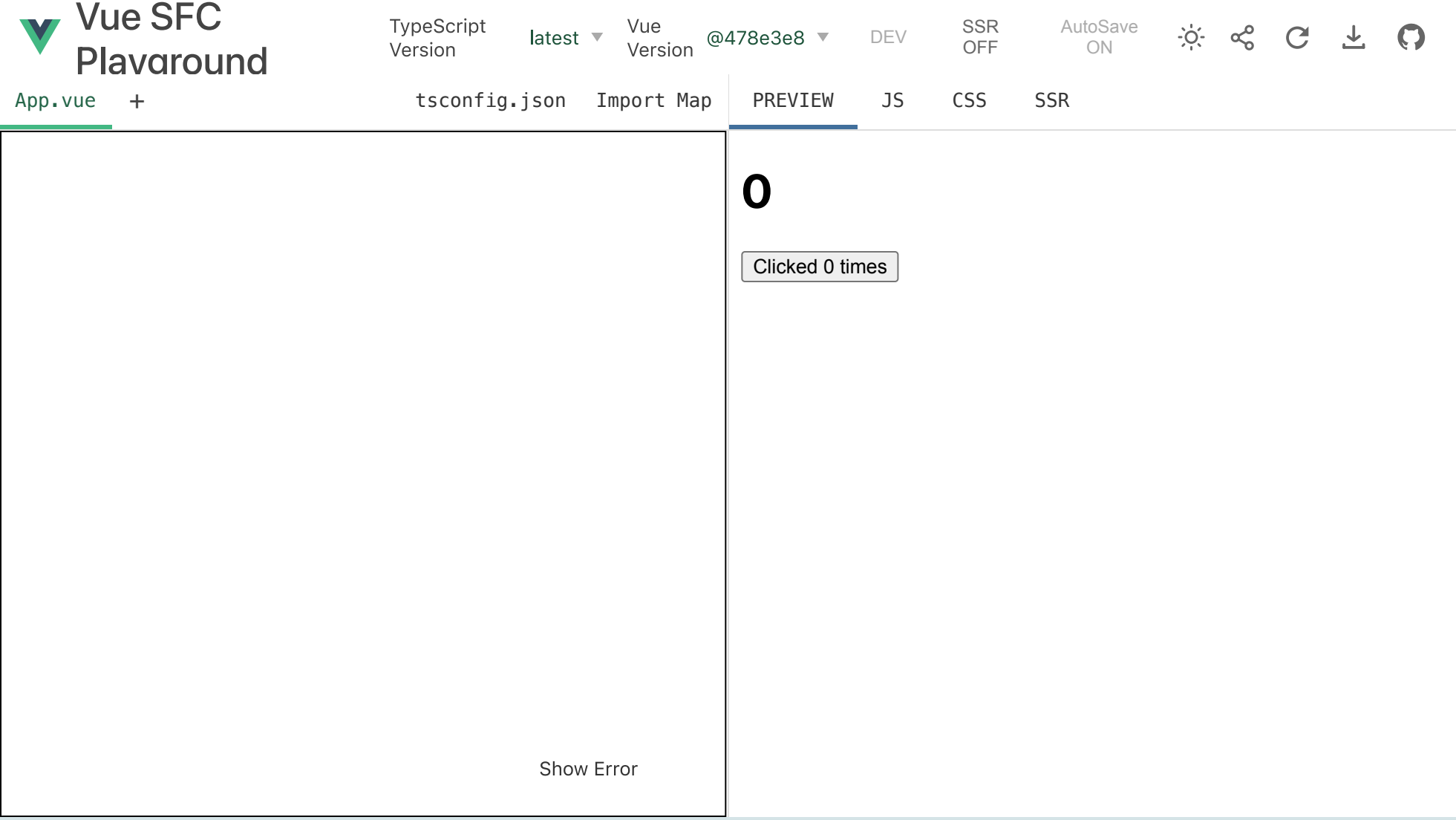
<style>

The appearance — colors & size

Vue Makes It Simple

```vue
<script setup>
import { ref } from 'vue'

const count = 0

function increment() {
  count += 1
}
</script>

<template>
  <h1>{{count}}</h1>
  <button @click="increment">
    Clicked {{ count }} times
  </button>
</template>
```




Vue SFC Playaround
TypeScript Version
latest
Vue Version
@478e3e8
DEV
SSR OFF
AutoSave ON
App.vue
+
tsconfig.json
Import Map
PREVIEW
JS
CSS
SSR
0
Clicked 0 times
Show Error

Environment Setup

Getting your Vue development environment ready on Jetstream2

Step 1 — Install VSCode

Skip this step if VSCode is already installed

- From Jetstream's web desktop interface:
  - Download the `.deb` package from the VSCode website
  - Launch the Terminal
  - `sudo apt install ./<vscode-downloaded-filename>.deb`

Step 2 — Install VSCode Extensions

Launch VSCode and install the following two extensions:

Vue - Official (Volar)

- Press `Ctrl+Shift+X` to open the Extensions panel
- Search for **Vue - Official**
- Click **Install**

Cline

- In the same Extensions panel, search for **Cline**
- Click **Install**

Step 3 — Configure Cline

Skip this step if Cline + Jetstream inference service was previously set up

Connect Cline to the Jetstream2 inference service running **llama-4-scout**:

1. Click the **Cline** icon in the Activity Bar (left sidebar)
2. Open Cline **Settings** (gear icon)
3. Set **API Provider** to `OpenAI Compatible`
4. Fill in the fields:

Field	Value
Base URL	`https://llm.jetstream-cloud.org/llama-4-scout/v1/`
Model	`llama-4-scout`

5. Click **Save**

Step 4 — Install nvm

`nvm` (Node Version Manager) lets you install and switch between Node.js versions.

Run this in your terminal:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh | bash
```

Then **close and reopen your terminal** (or run `source ~/.bashrc`) so the `nvm` command becomes available.

Verify the install:

```
nvm --version
```

Step 5 — Install Node.js

With `nvm` ready, install the latest version of Node.js:

```
nvm install node
```

Verify Node and npm are available:

```
node --version
npm --version
```

You should see version numbers printed for both — your environment is ready!

Clone & Run the Project

Step 1 — Clone the Repository

In your terminal, clone the workshop repository:

```
git clone https://github.com/chunlchan/flanker-task-demo
```

Then navigate into the project folder:

```
cd flanker-task-demo
```

Step 2 — Install Dependencies

Install the project's Node packages with npm:

```
npm install
```

This may take a moment the first time.

Step 3 — Start the Dev Server

```
npm run dev
```

You should see output like:

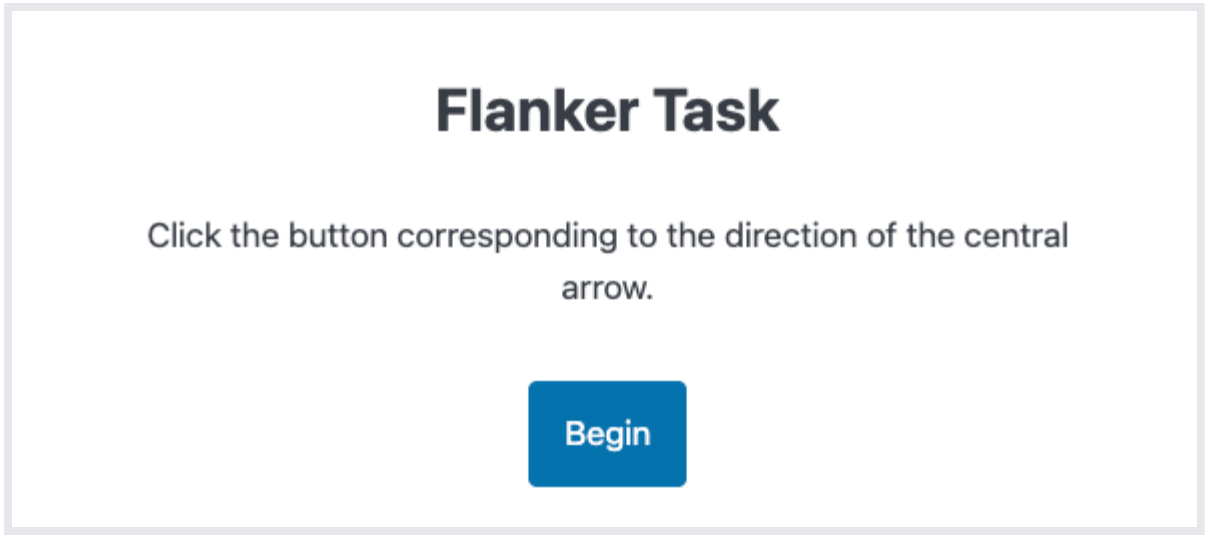
```
  VITE v5.x.x  ready in Xms

  ➜  Local:   http://localhost:5173/
```

Step 4 — Verify in the Browser

Open your browser and visit:

`http://localhost:5173`

If you see the Flanker page, your environment is fully set up and ready to go!



The Activity

Your Task — Add a Data Submit Feature

Using Cline in VSCode, you'll extend the Vue app to send data to a server.

Open the Cline chat panel and paste this prompt:

```
This app implements an Eriksen Flanker task. After a trial completes,
send the result as a JSON POST request to `https://chun-test.cis230103.projects.jetstream-cloud.org/flanker/submit.php`

{
  "responseTime": <trial response time in milliseconds, measured with performance.now()>,
  "correct": <true if the participant responded correctly to the flanker trial, false otherwise>,
  "team": <team name as a string, entered once by the user before trials begin>
}

"responseTime" and "correct" are already tracked by the app — wire
them into the payload after each trial. Add a single text input for
"team" that the user fills in before starting. Show a success message
when the server responds with 200, and an error message if the
request fails.
```

Cline will suggest the code changes — review them, then click **Accept**.

Verify It Works

- In a browser, visit `http://localhost:5173`
- Run through the Flanker task
- Per trial submissions should show up on the classroom dashboard

Deploy to Apache

Step 1 — Install & Start Apache

```
sudo apt install apache2
```

```
sudo systemctl enable apache2
sudo systemctl start apache2
```

Verify it’s running:

```
sudo systemctl status apache2
```

You should see `active (running)` in the output.

Step 2 — Build & Deploy the App

Build the production bundle:

```
npm run build
```

Copy the output to Apache’s web root:

```
sudo cp -r dist/* /var/www/html/
```

Step 3 — Find Your Server URL

1. Open the **Jetstream2 Exosphere** dashboard
2. Navigate to your instance → **Credentials** → **Hostname**
3. Copy the hostname — this is your public web server URL

Open it in a browser to confirm the app is live.

> *Note: If you are reusing the same Jetstream instance from Enis Afgan's session om Monday, you may need to run these commands to reset your IPtables:*

```
sudo iptables -t nat -D PREROUTING -p tcp --dport 80 -j REDIRECT --to-port 8080
sudo iptables -t nat -D OUTPUT -p tcp --dport 80 -j REDIRECT --to-port 8080
```

Step 4 — Share With Your Team

- Post your URL in **Slack** and have your teammates open it in their browser.
- If they can load the app and submit a Flanker trial, you've successfully completed the activity!

Bonus Activities

If you have extra time, give these a try!

Bonus 1 — Restyle the Flanker Stimuli

Use Cline to swap out the `>` and `<` arrows for something new. Try prompting:

```
Replace the flanker arrow stimuli (> and <) with emoji arrows (➡️ and ⬅️).
```

If you are feeling adventurous, try using completely different visual cues such as letters or numbers. The same principle applies with the middle item being the one in focus while the "flanking" items should be ignored.

```
XXAXX, AAXAA, AAAAA, XXXXX
```

Think about it: If you move away from directional arrows, what else in the app might need to change?

- Does the correctness logic still hold with the new stimuli?
- Does anything in the UI labels or instructions need updating?

Bonus 2 — Compare AI Models

Part 1: Prep

- Create a new folder to start a brand new blank project space

```
cd /path/to/your_new_folder
npm create vue@latest
```

- Project name: *Your choice*
- Use TypeScript: *No*
- Select features to include in your project: *(None selected, just press enter to continue)*
- Select experimental features to include in your project: *(None selected, just press enter to continue)*
- Skip all example code and start with a blank Vue project: *Yes*

```
cd your_project_name
npm install
npm run dev
```

Bonus 2 — Compare AI Models

Part 2: Run Inference on Multiple Models

Using the full project plan, try building the app with different models and compare the results.

Copy and paste the plan into Cline. Then rollback using Cline's checkpoint feature, switch to different model and rerun the same plan.

Ask yourself:

- Does every model produce a working application, or do some introduce bugs?
- Are some models faster at generating code?
- Do the outputs differ in style, structure, or completeness?
- Which model would you reach for on a real project?

In Conclusion

- Web development for research can be relatively simple.
- Complexity usually comes in the form of integrating with other services.
- Tools that are worth getting familiar with. These are pretty common across academic institutions.
  - Crowd Sourcing: Prolific
  - Consents, Surveys: Qualtrics
  - RedCap: Clinical trials, longitudinal studies
  - Globus: Secure data transfer

Keep Making Connections!